REPÚBLICA PORTUGUESA
EDUCAÇÃO, CIÊNCIA E INOVAÇÃO

AEGP
AGRUPAMENTO DE ESCOLAS GABRIEL PEREIRA - ÉVORA

AGRUPAMENTO DE ESCOLAS GABRIEL PEREIRA - ÉVORA

Programação e Sistemas de Informação

MÓDULO 14

LINGUAGENS DE MANIPULAÇÃO DE DADOS

CURSO PROFISSIONAL
TÉCNICO DE GESTÃO E PROGRAMAÇÃO DE SISTEMAS INFORMÁTICOS
PROGRAMAÇÃO E SISTEMAS DE INFORMAÇÃO

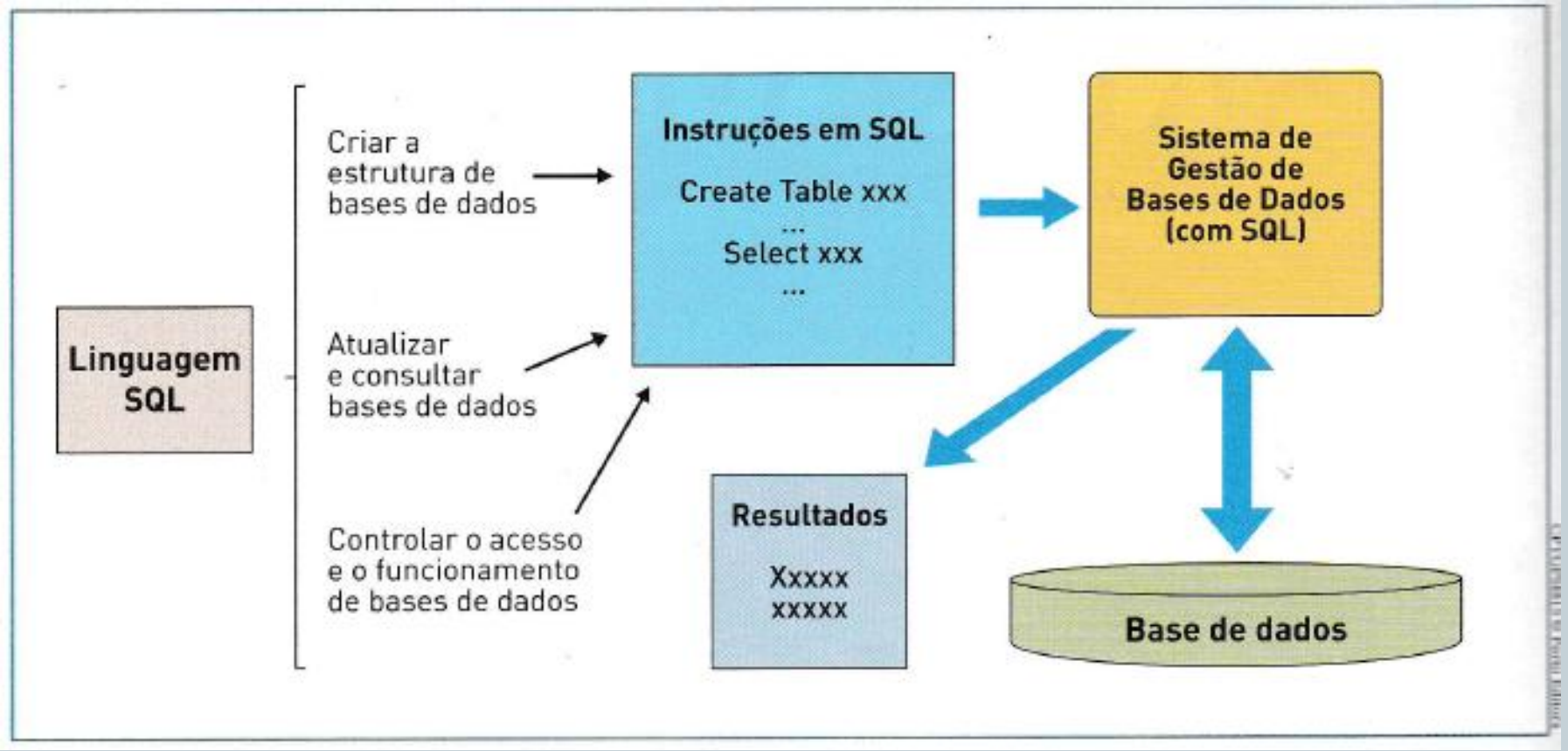
Linguagens de Manipulação de Dados

STRUCTURED QUERY LANGUAGE (SQL)

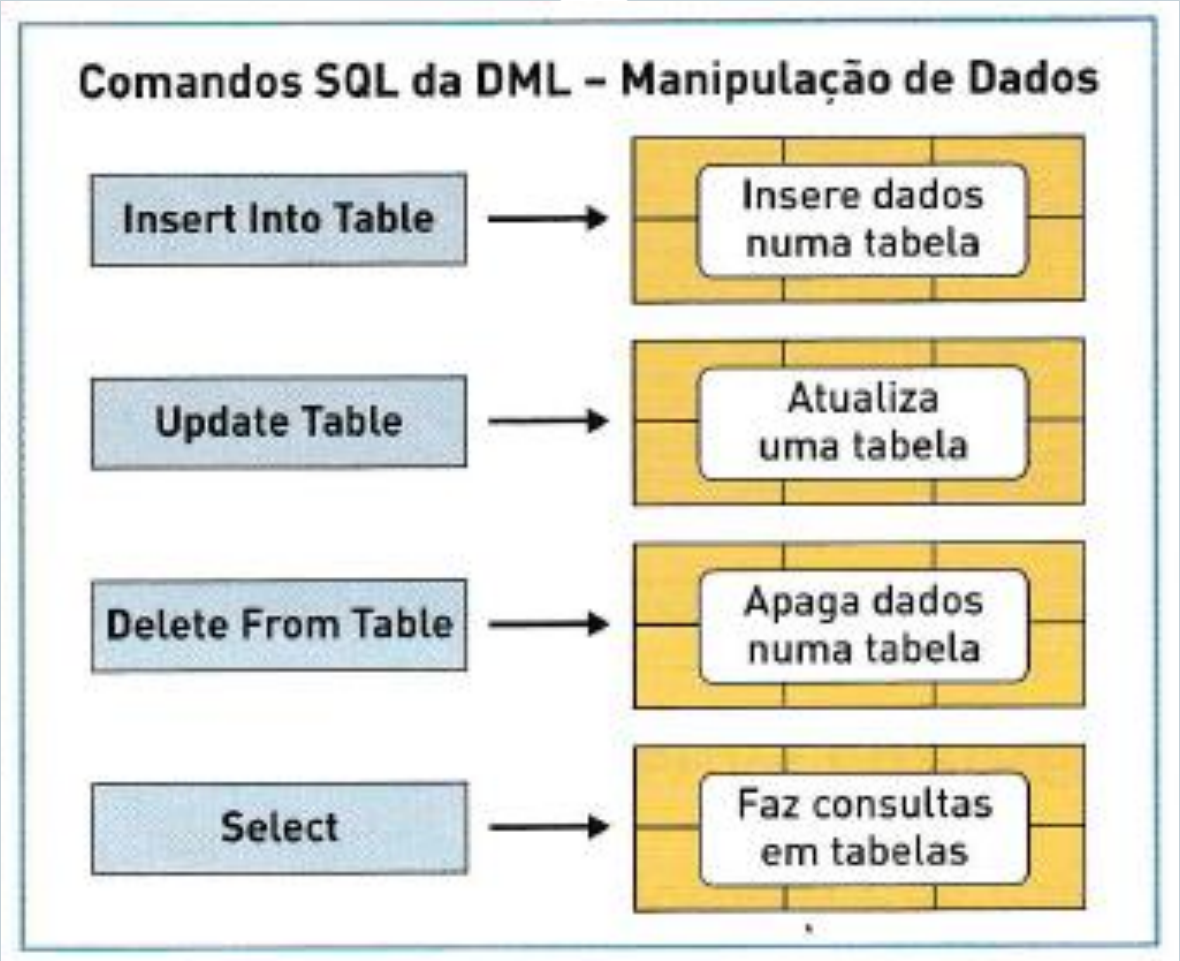
SQL – Structured Query Language

- Desenvolvida originalmente no início dos anos 70 nos laboratórios da IBM em San José, dentro do projeto System R, que queria demonstrar a viabilidade da implementação do modelo de dados relacional proposto por E. F. Codd;
- O nome original era SEQUEL (Structured English Query Language);
- Foi criado um padrão para a linguagem pela American National Standards Institute (ANSI) em 1986 e ISO (International Organization for Standardization) em 1987.

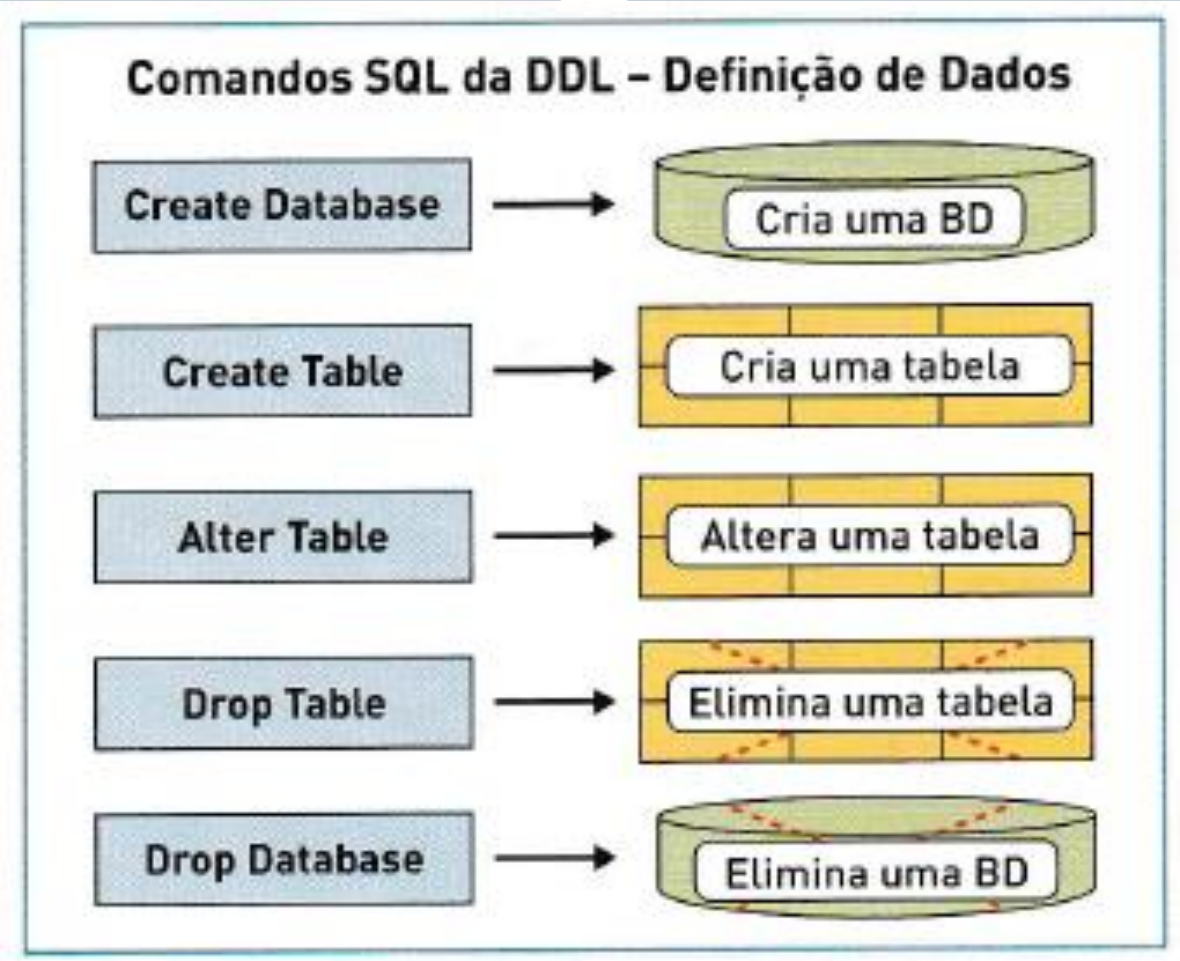
SQL – Structured Query Language



SQL - DML

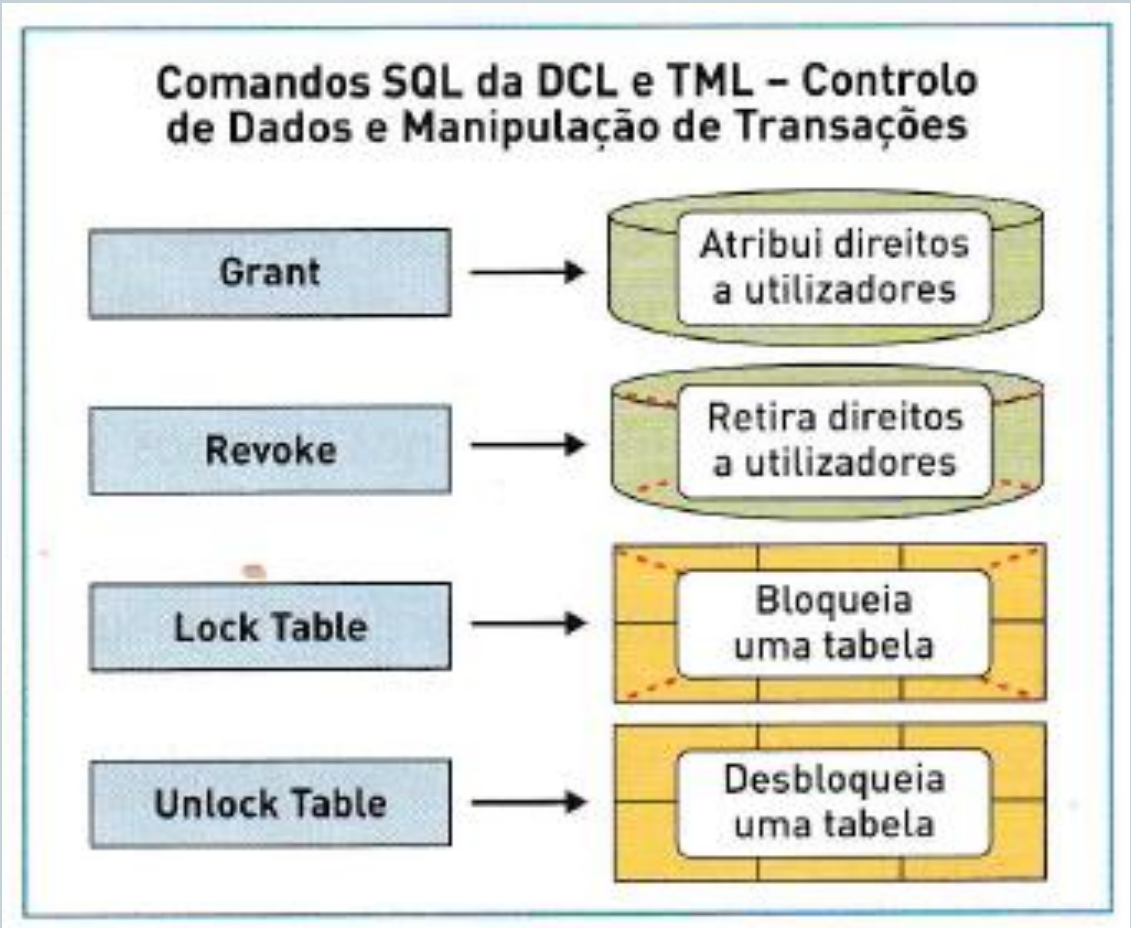

Comandos SQL da DML – Manipulação de Dados
Insert Into Table
Insere dados numa tabela
Update Table
Atualiza uma tabela
Delete From Table
Apaga dados numa tabela
Select
Faz consultas em tabelas

SQL – DDL



(Módulo 15)

SQL – DCL e TML



(Módulo 15)

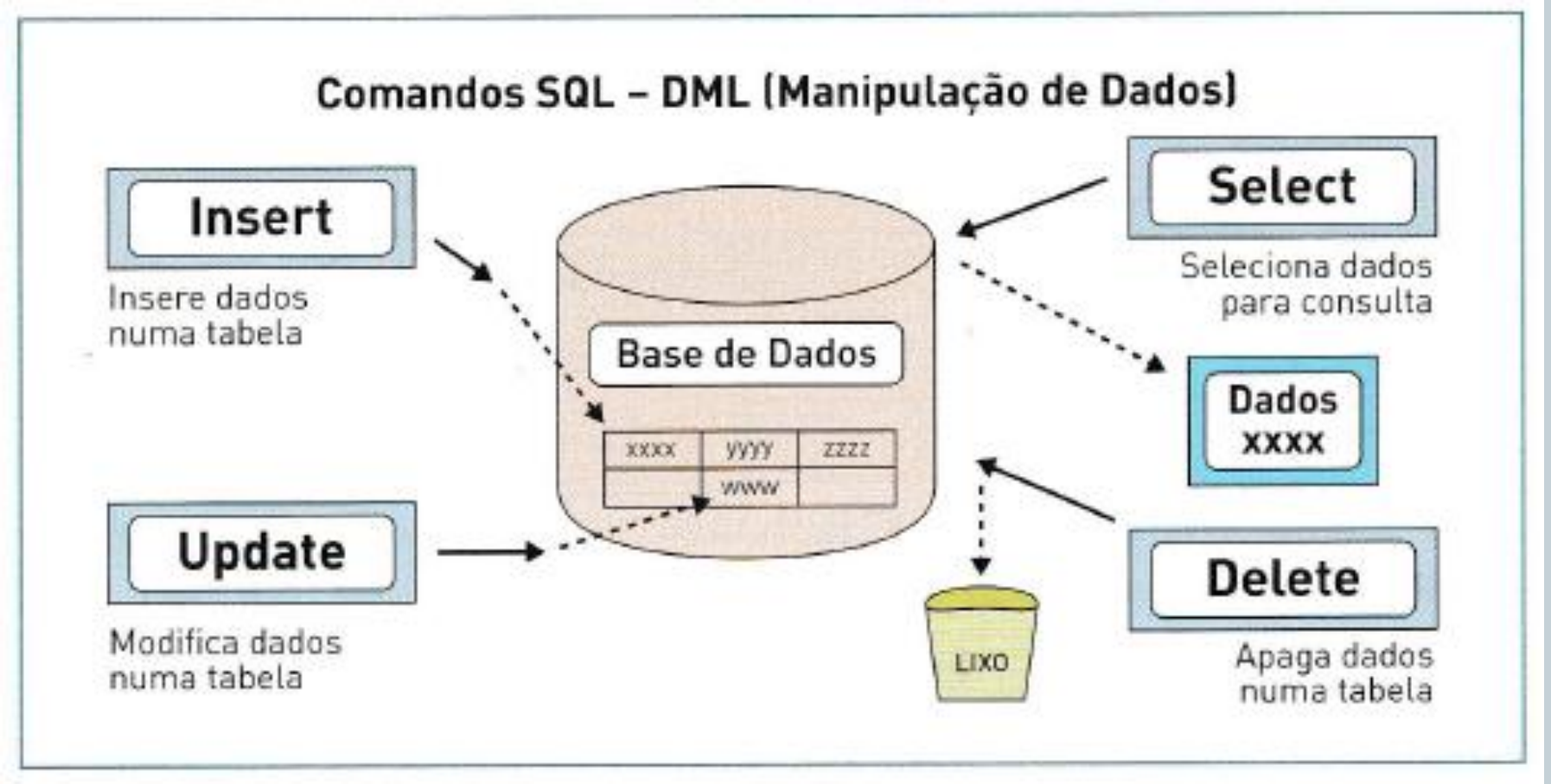
Linguagens de Manipulação de Dados

DATA MANIPULATION LANGUAGE (DML)

SQL - DML

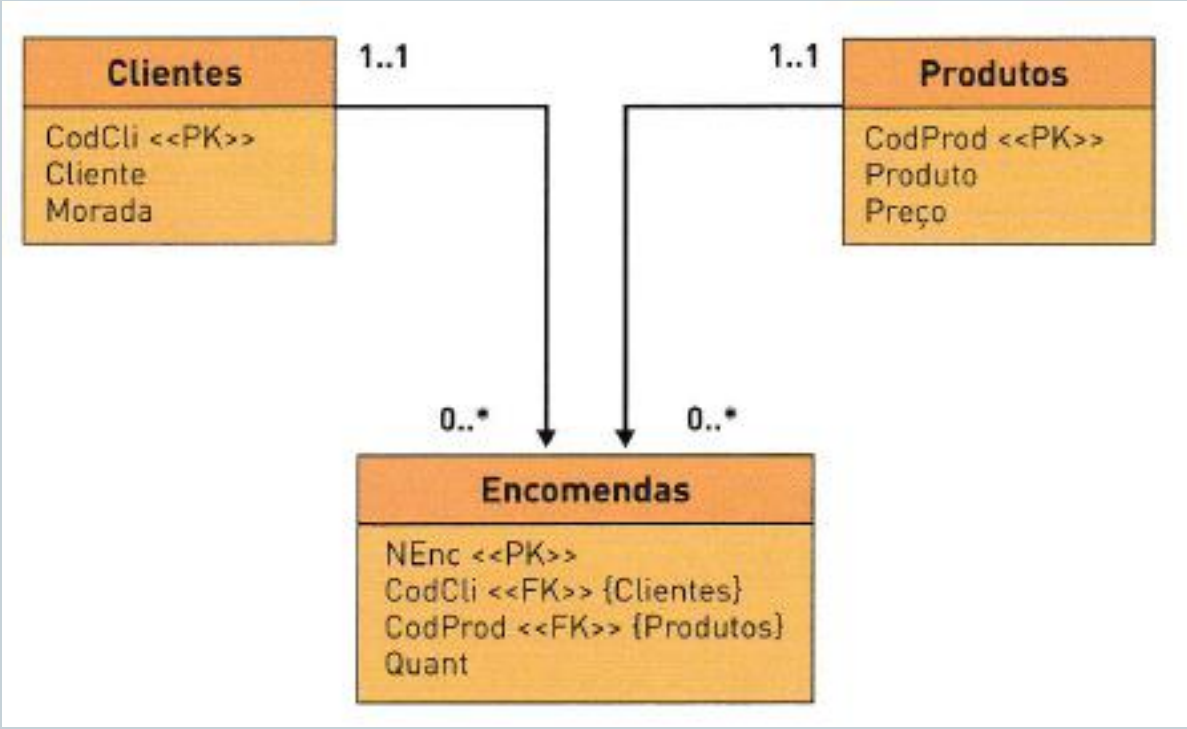
- Linguagem standard de manipulação de dados em sistema de gestão de bases de dados;
- Permite a pesquisa, a inserção, a alteração e a remoção de registos numa base de dados;
- Linguagem “universal” para pesquisas em bases de dados.

SQL - DML


Comandos SQL – DML (Manipulação de Dados)
Insert
Insere dados numa tabela
Update
Modifica dados numa tabela
Base de Dados
xxxx
yyyy
zzzz
www
Select
Seleciona dados para consulta
Dados xxxx
Delete
Apaga dados numa tabela
LIXO

SQL – DML

- **Atualizar uma base de dados**
  - **Inserir dados**
  - **Alterar dados**
  - **Apagar dados**



SQL – DML

- **Inserir Dados**
  - **Indicar os nomes dos campos e os valores;**
  - **O campo CodCli não necessita de valores porque está definido como auto-incremento.**

```
Insert into Clientes
(Cliente, Morada)
Values
("Aníbal", "Lisboa"),
("Belmiro", "Braga"),
("Casimiro", "Coimbra");
```

CodCli	Cliente	Morada
1	Aníbal	Lisboa
2	Belmiro	Braga
3	Casimiro	Coimbra

SQL – DML

- **Inserir Dados**
  - **O campo CodProd é auto-incremento;**
  - **Os valores do tipo String podem ser incluídos dentro de aspas (“Lisboa”) ou apostrofe (‘Berbequim’).**

```
Insert into Produtos
Values
(Null, 'Portátil', 400),
(Null, 'Berbequim', 50),
(Null, 'MS Office', 150);
```



CodProd	Produto	Preço
1	Portátil	400
2	Berbequim	50
3	MS Office	150

SQL – DML

- **Inserir Dados**
  - **O campo NEnc é auto-incremento;**
  - **Os campos CodCli e CodProd são chave estrangeira, por isso os valores têm de existir nas respetivas tabelas.**

```
Insert into Encomendas
Values
(Null, 1,1,10),
(Null, 2,1,2),
(Null, 3,2,5)
(Null, 1,3,1);
```


NEnc	CodCli	CodProd	Quant
1	1	1	10
2	2	1	2
3	3	2	5
4	1	3	1

SQL – DML

- **Apagar Registos**
  - **O comando Delete apaga registos (linhas inteiras numa tabela;**
  - **A sua sintaxe é a seguinte:**

```
Delete From nome_tabela
[Where condição];
```

  - **Os parêntesis retos significam que a cláusula Where é opcional.**

SQL – DML

- **Apagar Registos**

```
Delete From Produtos;
```

  - **Apaga todos os dados da tabela Produtos (não tem cláusula Where).**

SQL – DML

- **Apagar Registos**

Tabela Produtos antes

CodProd	Produto	Preço
1	Portátil	400
2	Berbequim	50
3	MS Office	150

Exemplo comando Delete:

```
Delete From Produtos
Where Produto = "MS Office";
```

Tabela Produtos depois

CodProd	Produto	Preço
1	Portátil	400
2	Berbequim	50

CPTG|PSI|8|5 © Porto Editora

SQL – DML

- **Alterar dados**
  - **A alteração de informação faz-se com o comando Update;**
  - **A sua sintaxe é a seguinte:**

```
Update nome_tabela
Set campo1 = expressão1
[, campo2 = expressão2] [, ...]
[Where condição];
```

SQL – DML

- **Alterar dados**

```
Update Produtos
Set Preco = Preco * 1.05 ;
```

  - **Altera o valor do preço de todos os produtos para um valor multiplicado por 1,05;**
  - **Por exemplo, 10€ passam a 10,50€.**

SQL – DML

- **Alterar dados**

Tabela Produtos antes

CodProd	Produto	Preço
1	Portátil	400
2	Berbequim	50
3	MS Office	150

Exemplo comando Update

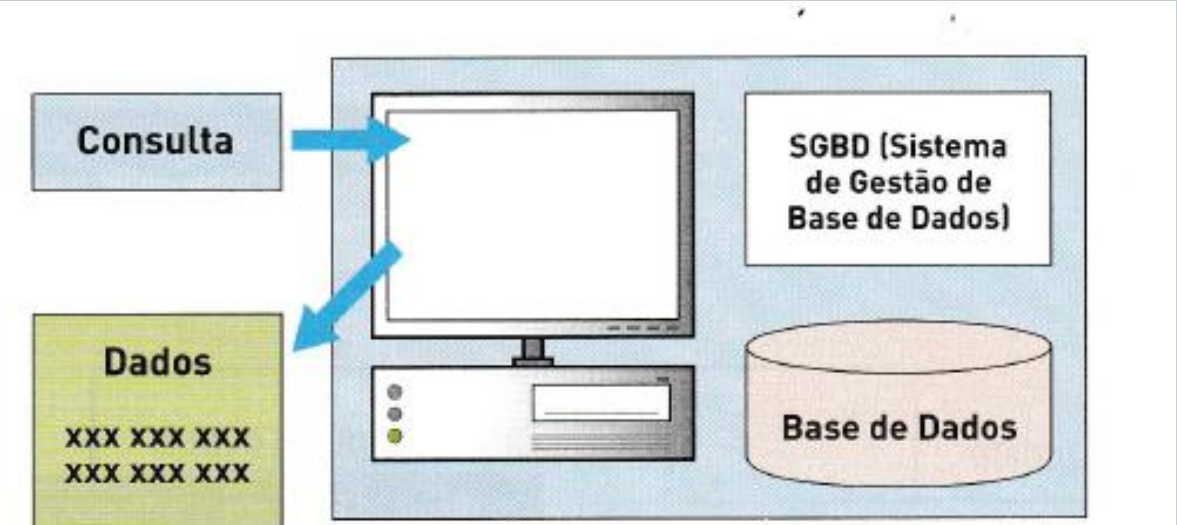
```
Update Produtos
Set Preco = 350
Where Produto = "Portátil";
```

Tabela Produtos depois

CodProd	Produto	Preço
1	Portátil	350
2	Berbequim	50
3	MS Office	150

SQL – DML

- **Consultar dados**
  - **O comando Select permite consultar a informação guardada na BD**



O motor do SGBD (*database engine*) recebe, interpreta e processa a consulta (*query*) e devolve os dados correspondentes à pesquisa.

SQL – DML

- **Consultar informação**
  - **A sintaxe do comando Select pode ser muito complexa, porque existem muitas possibilidades de conjugar a informação das tabelas;**
  - **Esta é uma versão simplificada da estrutura.**

```
Select lista_de_campos
From tabela(s)
[Where condição]
[Group By lista_de_campos]
[Having condição]
[Order By lista_de_campos]
```

SQL – DML

- **Consultar informação**
  - **Consultar todos os campos de todos os registos da tabela:**

```
Select * From tabela;
```

Exemplo comando Select

```
Select *
From Produtos;
```

Resultado da consulta

CodProd	Produto	Preco
1	Portátil	400
2	Berbequim	50
3	MS Office	150

SQL – DML

- **Consultar informação**

Exemplo comando Select

```
Select *
From Produtos
Where Preco > 100;
```

Resultado da consulta

CodProd	Produto	Preco
1	Portátil	400
3	MS Office	150

  - **Consultar todos os campos dos registos da tabela Produtos onde o Preço é maior que 100.**

SQL – DML

- **Consultar informação**

Exemplo comando Select

```
Select Produto,
Preco
From Produtos
Where Preco > 100;
```

Resultado da consulta

Produto	Preco
Portátil	400
MS Office	150

  - **Consultar apenas o nome e o preço dos registos da tabela Produtos onde o Preço é maior que 100.**

SQL – DML

- **Operadores utilizados na cláusula Where**

Operadores de comparação	
Operador	**Descrição**
=	Igual a
<	Menor que
>	Maior que
<=	Menor ou Igual
>=	Maior ou Igual
<>	Diferente de

Outros Operadores	
Operador	**Descrição**
Between	Entre
In	Dentro de
Like	Igual a
Is [Not] Null	É [não é] valor nulo

Operadores Lógicos	
Operador	**Descrição**
And	E [conjunção]
Or	Ou [disjunção]
Not	Não [negação]

SQL – DML

- **Tabela de Produtos com mais alguns dados**

CodProd	Produto	Preco
1	Portátil	400
2	Berbequim	50
3	MS Office	150
4	Impressora	100
5	Portátil	200
6	Impressora	50

SQL – DML

- **Consultar informação**
  - **Consultar os registos cujo Preço está entre 200 e 400, inclusive:**

```
Select * From Produtos
Where Preco >= 200 And Preco <= 400;
```

O resultado desta consulta seria:

CodProd	Produto	Preco
1	Portátil	200
5	Portátil	400

Esta consulta também poderia ser feita assim:

```
Select * From Produtos
Where Preco Between 200 And 400;
```

SQL – DML

- **Consultar informação**
  - **Analise o que faz esta consulta:**

```
Select * From Produtos
Where Preco < 100 Or Preco > 200;
```

O resultado desta consulta seria:

CodProd	Produto	Preco
1	Portátil	400
2	Berbequim	50
6	Impressora	50

SQL – DML

- **Consultar informação**
  - **O resultado é o mesmo?**

```
Select * From Produtos
Where Preco = 100 Or Preco = 200;
```

O resultado desta consulta seria:

CodProd	Produto	Preco
4	Impressora	100
5	Portátil	200

```
Select * From Produtos
Where Preco In (100, 200);
```

SQL – DML

- **Consultar informação**

CodProd	Produto	Preco
1	Portátil	400
2	Berbequim	50
3	MS Office	150
4	Impressora	100
5	Portátil	200
6	Impressora	50

Consulta

```
Select * From Produtos
Where
Produto Like "P%";
```

Resultado da consulta

CodProd	Produto	Preco
1	Portátil	400
5	Portátil	200

- **Consultar os registos cujo nome começa por P;**
- **% equivale a um conjunto de caracteres;**
- **? equivale a um caracter;**

SQL – DML

- **Consultar informação**
  - **Como podemos descrever esta consulta?**

```
Select * From Produtos
Where Produto Like "%Office%";
```

O resultado desta consulta seria:

CodProd	Produto	Preco
3	MS Office	150

SQL – DML

- **Ordenação de dados em Consultas**
  - Utiliza-se a cláusula Order By;
  - ASC para ascendente e DESC para descendente;
  - Por omissão a ordenação é ascendente;
  - A ordenação no campo2 é feita "dentro" da ordenação do campo1.

```
Select ... From ...
[...]
Order By campo1 [Asc|Desc], campo2...
```

SQL – DML

- **Ordenação de dados em Consultas**
  - **Seleção dos registos da tabela Produtos ordenados por Preço descendente**

```
Select * From Produtos
Order By Preco Desc;
```

CodProd	Produto	Preco
1	Portátil	400
5	Portátil	200
3	MS Office	150
4	Impressora	100
6	Impressora	50
2	Berbequim	50

  - **Seleção dos registos da tabela Produtos em que o Preço é superior a 200 ordenados por nome ascendente (ordem alfabética)**

```
Select * From Produtos
Where Preco > 200
Order By Produto ;
```

SQL – DML

- **Funções de cálculos e agregações em Consultas**

Função	Significado
Count	Conta o número de registos encontrados numa consulta
Sum	Soma de todos os valores para o campo indicado
Avg	Média (average) de todos os valores para o campo indicado
Max	Máximo valor no campo indicado
Min	Mínimo valor no campo indicado
First	O primeiro de um conjunto ordenado de valores
Last	O último de um conjunto ordenado de valores

SQL – DML

- **Função Count**
  - **A partícula As permite atribuir um nome à coluna do resultado;**

```
Select Count(*) As Total
From Produtos ;
```

Devolve o número de linhas da tabela Produtos.

Resultado:

Total
6

```
Select Count(Distinct Preco)
From Produtos;
```

Devolve o número de preços distintos (Distinct) no campo Preco. Neste caso, o resultado seria: 5 (o preço 50 só é contado uma vez).

SQL – DML

- **Funções Avg (média) e Sum (soma)**

CodProd	Produto	Preco
1	Portátil	400
2	Berbequim	50
3	MS Office	150
4	Impressora	100
5	Portátil	200
6	Impressora	50

Consulta

```
Select Avg(Preco)
As Media
From Produtos;
```

Resultado da consulta

Media
158.33

```
Select Sum(Preco)
From Produtos
Where Produto Like "Portátil";
```

SQL – DML

- **Funções Min (mínimo) e Max (soma)**

CodProd	Produto	Preco
1	Portátil	400
2	Berbequim	50
3	MS Office	150
4	Impressora	100
5	Portátil	200
6	Impressora	50

Consulta

```
Select Min(Preco)
From Produtos
Where Produto Like "Impressora";
```

Resultado

Min(Preco)
50

```
Select Max(Preco)
From Produtos;
```

SQL – DML

- **Agrupamentos em consultas com a cláusula Group By**
  - **Para agrupar dados que dados que contenham características semelhantes;**
  - **Usada em conjugação com as funções de agregação (Count, Sum, Avg, Max e Min);**
  - **Obter uma listagem com o nome do Produto e de quantos registos existem de cada:**

```
Select Produto, Count(*)
From Produtos
Group By Produto;
```

SQL – DML

- **Agrupamentos em consultas com a cláusula Group By**

CodProd	Produto	Preco
1	Portátil	400
2	Berbequim	50
3	MS Office	150
4	Impressora	100
5	Portátil	200
6	Impressora	50

Consulta

```
Select Produto, Count(*)
From Produtos
Group By Produto;
```

Resultado

Produto	Count(*)
Berbequim	1
Impressora	2
MS Office	1
Portátil	2

CIF©Porto Editora

SQL – DML

- **Agrupamentos em consultas com a cláusula Group By**
  - **Qual o preço mais elevado dentro de cada grupo de produtos com o mesmo nome?**

CodProd	Produto	Preco
1	Portátil	400
2	Berbequim	50
3	MS Office	150
4	Impressora	100
5	Portátil	200
6	Impressora	50

Consulta

```
Select Produto, Max(Preco)
From Produtos
Group By Produto;
```

Resultado

Produto	Max(Preco)
Berbequim	50
Impressora	100
MS Office	150
Portátil	400

SQL – DML

- **Agrupamentos em consultas com a cláusula Having**
  - **Podemos acrescentar Having à cláusula Group By para selecionar os registos por uma condição;**
  - **Semelhante à cláusula Where mas adequada para grupos de registos.**

Consulta

```
Select Produto, Max(Preco)
From Produtos
Group By Produto
Having Max(Preco) > 100;
```

Resultado

Produto	Max(Preco)
MS Office	150
Portátil	400

SQL – DML

- **Funções de Linha**
  - LCASE (lower case - minúsculas)
  - UPPER (upper case – maiúsculas)
  - INITCAP (“escola secundária Dom") = Escola Secundária Dom (ORACLE)
  - CONCAT ("String1", "String2") = String1String2
  - SUBSTR ("String", 1, 3) = Str
  - LENGTH (“abc") = 3
  - IFNULL (SAL, 0) = Se SAL for NULO seu valor será convertido para zero.
  - ROUND (78.735, 2) = 78.74
  - ROUND (78.735, 0) = 79
  - TRUNC (78.735, 2) = 78.73
  - TRUNC (78.735) = 78
  - MOD (7, 2) = 1 (resto da divisão)

SQL – DML

- **Funções de Linha**
  - **MID() - Extract characters from a text field**
  - **STR() – converte um número em testo**

    SELECT STR(21,34); - resultado: 21 – retira as casas decimais

    SELECT STR(21,34, 5, 2) – resultado: 21,34 – converte com as casas decimais

    5 é o comprimento total da string.

    2 é o número de casas decimais.
  - **LEN() - comprimento de um texto**
  - **GETDATE() – data e hora do sistema**
  - **FORMAT() – formata um campo**

SQL – DML

- **Funções para tipo de dados Data**
  - **DATEPART(parte, data) - retorna uma parte específica de uma data, como ano, mês, dia ou hora.**
  - **DATEADD(parte, quantidade, data) - adiciona um intervalo de tempo a uma data (ou subtrai).**

Parte da data	Significado
year ou yy	Ano
month ou mm	Mês
day ou dd	Dia
Hour ou hh	Hora
Minute ou mm	Minuto
Second ou ss	Segundo
Weekday ou dw	Dia da semana
quarter	Trimestre

Exemplos:

```
SELECT
    DATEPART(year, '2021-09-07') AS Ano,
    DATEPART(month, '2021-09-07') AS Mes,
    DATEPART(day, '2021-09-07') AS Dia;

SELECT DATEADD(day, 10, '2021-09-07') AS NovaData;
```

SQL – DML

- **Consultas com subqueries**
  - O comando Select pode ser inserido dentro de outro Select, ao que se dá o nome de subquery;
  - Como saber qual ou quais os produtos que têm o preço mais baixo?
  - Para saber o preço mais baixo:

```
Select Min(Preco)
From Produtos;
```

com o seguinte resultado:

Min(Preco)
50

SQL – DML

- **Consultas com subqueries**
  - **Para saber os produtos que têm o preço igual ao mínimo:**

Consulta

```
Select * From Produtos
Where Preco = (Select Min(Preco)
               From Produtos);
```

Resultado

CodProd	Produto	Preco
2	Berbequim	50
6	Impressora	50

SQL – DML

- **Consultas com subqueries**
  - **Também podemos cruzar dados entre duas tabelas, embora existam outras formas;**
  - **Quais as encomendas que temos de Portáteis?**

Produtos

CodProd	Produto	Preco
1	Portátil	400
2	Berbequim	50
3	MS Office	150
4	Impressora	100
5	Portátil	200
6	Impressora	50

Encomendas

NEnc	CodCli	CodProd	Quant
1	1	1	10
2	2	1	2
3	3	2	5
4	1	3	1

```
Select * From Encomendas
Where CodProd In (Select CodProd
                  From Produtos
                  Where Produto = 'Portátil');
```

Resultado

NEnc	CodCli	CodProd	Quant
1	1	1	10
2	2	1	2

SQL – DML

- **Consultas relacionais**
  - **Há 8 operações possíveis de aplicar sobre os dados das tabelas:**
    - **projeção,**
    - **restrição,**
    - **união,**
    - **interseção,**
    - **diferença,**
    - **produto,**
    - **divisão,**
    - **junção.**

SQL – DML

- **Consultas relacionais**

Consulta global

```
Select * From Clientes;
```

CodCli	Cliente	Morada
1	Aníbal	Lisboa
2	Belmiro	Braga
3	Casimiro	Coimbra

Consulta de projeção

```
Select Cliente, Morada From Clientes;
```

Cliente	Morada
Aníbal	Lisboa
Belmiro	Braga
Casimiro	Coimbra

Consulta de restrição

```
Select * From Clientes
Where Morada Like "Lisboa";
```

CodCli	Cliente	Morada
1	Aníbal	Lisboa

Consulta de projeção e restrição

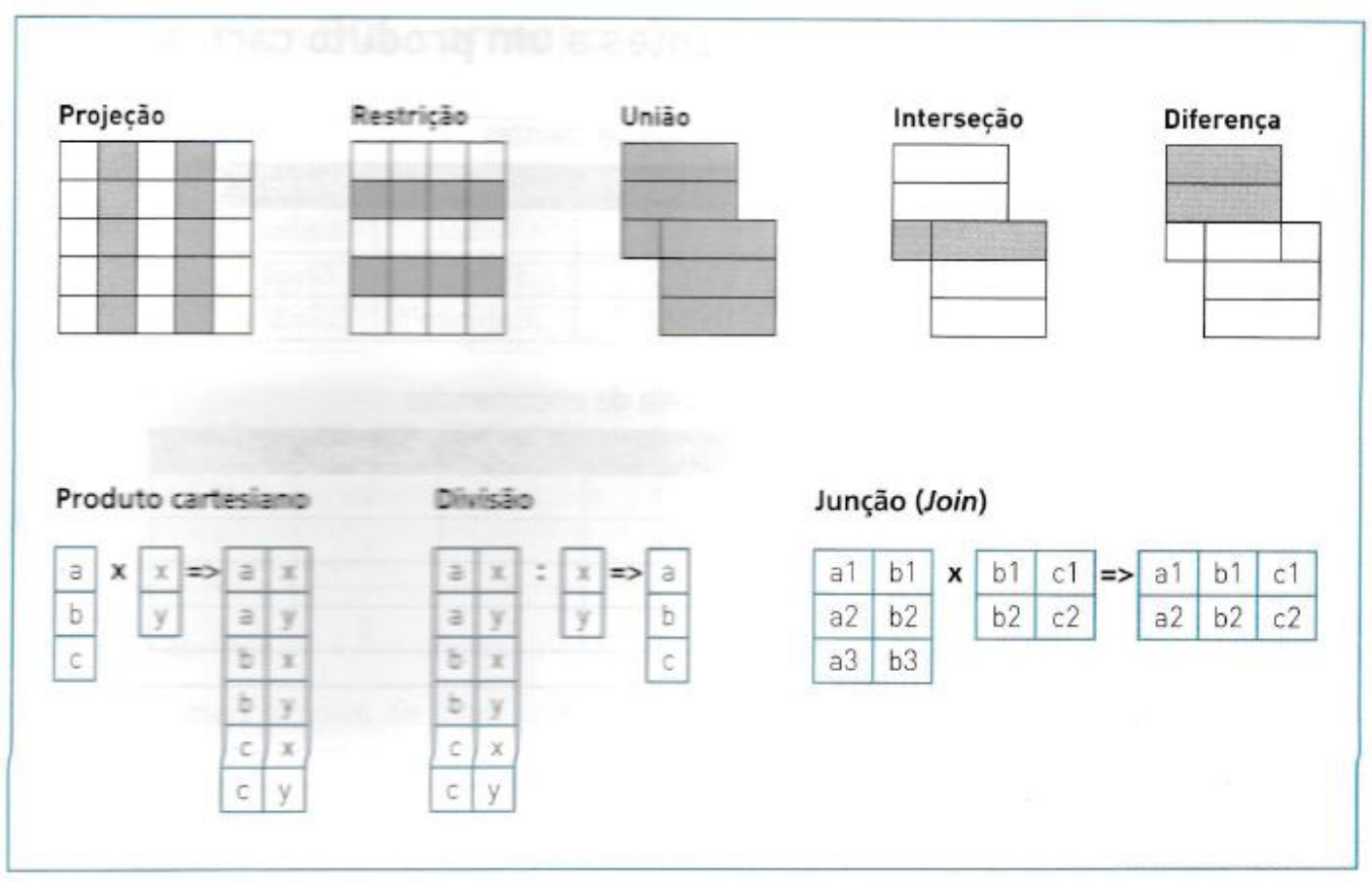
```
Select Cliente, Morada
From Clientes Where Morada Like "Lisboa";
```

Cliente	Morada
Aníbal	Lisboa

CPTGPS1815 © Porto Editora

SQL – DML

- **Consultas relacionais**



SQL – DML

- **Consultas correspondente a um produto cartesiano**

Tabela de clientes

CodCli	Cliente	Morada
1	Aníbal	Lisboa
2	Belmiro	Braga
3	Casimiro	Coimbra

Tabela de encomendas

NEnc	CodCli	CodProd	Quant
1	1	1	10
2	2	1	2
3	3	2	5
4	1	3	1

```
Select *
From Clientes, Encomendas;
```

Resultado – Produto cartesiano

CodCli	Cliente	Morada	NEnc	CodCli	CodProd	Quant
1	Aníbal	Lisboa	1	1	1	10
1	Aníbal	Lisboa	2	2	1	2
1	Aníbal	Lisboa	3	3	2	5
1	Aníbal	Lisboa	4	1	3	1
2	Belmiro	Braga	1	1	1	10
2	Belmiro	Braga	2	2	1	2
2	Belmiro	Braga	3	3	2	5
2	Belmiro	Braga	4	1	3	1
3	Casimiro	Coimbra	1	1	1	10
3	Casimiro	Coimbra	2	2	1	2
3	Casimiro	Coimbra	3	3	2	5
3	Casimiro	Coimbra	4	1	3	1

CPTC/PSI/15 © Porto Editora

SQL – DML

- **Consultas correspondente a uma junção**
  - **Equi-join**

Tabela de clientes

CodCli	Cliente	Morada
1	Aníbal	Lisboa
2	Belmiro	Braga
3	Casimiro	Coimbra

Tabela de encomendas

NEnc	CodCli	CodProd	Quant
1	1	1	10
2	2	1	2
3	3	2	5
4	1	3	1

```
Select *
From Clientes, Encomendas;
Where
Clientes.CodCli = Encomendas.CodCli;
```

Resultado – Equi-join

CodCli	Cliente	Morada	NEnc	CodCli	CodProd	Quant
1	Aníbal	Lisboa	1	1	1	10
1	Aníbal	Lisboa	4	1	3	1
2	Belmiro	Braga	2	2	1	2
3	Casimiro	Coimbra	3	3	2	5

SQL – DML

- **Consultas correspondente a uma junção**
  - **inner-join**

Tabela de clientes

CodCli	Cliente	Morada
1	Aníbal	Lisboa
2	Belmiro	Braga
3	Casimiro	Coimbra

Tabela de encomendas

NEnc	CodCli	CodProd	Quant
1	1	1	10
2	2	1	2
3	3	2	5
4	1	3	1

```
Select *
From Clientes inner join Encomendas On
Clientes.CodCli = Encomendas.CodCli;
```

Resultado – Inner join

CodCli	Cliente	Morada	NEnc	CodCli	CodProd	Quant
1	Aníbal	Lisboa	1	1	1	10
1	Aníbal	Lisboa	4	1	3	1
2	Belmiro	Braga	2	2	1	2
3	Casimiro	Coimbra	3	3	2	5

CPTUP5815 © Porto Editora

SQL – DML

- **Consultas correspondente a uma junção**
  - **outer-join**
    - **left-join**
      - **são apresentados todos os registos da tabela considerada em primeiro lugar (à esquerda), com a informação correspondente da tabela da direita;**
    - **Right-join**
      - **são apresentados todos os registos da tabela considerada em segundo lugar (à direita), com a informação correspondente da tabela da esquerda;**

SQL – DML

- **Consultas correspondente a uma junção**
  - **left-join**

Produtos

CodProd	Produto	Preço
1	Portátil	400
2	Berbequim	50
3	MS Office	150
4	Impressora	100
5	Portátil	200
6	Impressora	50

Tabela Encomendas

NEnc	CodCli	CodProd	Quant
1	1	1	10
2	2	1	2
3	3	2	5
4	1	3	1

```
Select *
From Produtos left join Encomendas On
Produtos.CodProd = Encomendas.CodProd;
```

Resultado – Left join

CodProd	Produto	Preço	NEnc	CodCli	CodProd	Quant
1	Portátil	400	1	1	1	1
1	Portátil	400	2	2	1	2
2	Berbequim	50	3	3	2	5
3	MS Office	150	4	1	4	1
4	Impressora	100	Null	Null	Null	Null
5	Portátil	200	Null	Null	Null	Null
6	Impressora	50	Null	Null	Null	Null

SQL – DML

- **Consultas com três (ou mais) tabelas**

Tabela Clientes

CodCli	Cliente	Morada
1	Aníbal	Lisboa
2	Belmiro	Braga
3	Casimiro	Coimbra

Tabela Produtos

CodProd	Produto	Preco
1	Portátil	400
2	Berbequim	50
3	MS Office	150
4	Impressora	100
5	Portátil	200
6	Impressora	50

Tabela Encomendas

NEnc	CodCli	CodProd	Quant
1	1	1	10
2	2	1	2
3	3	2	5
4	1	3	1

```
Select Cliente, Produto, Quant
From Clientes, Produtos, Encomendas
Where Clientes.CodCli = Encomendas.CodCli
And
Produtos.CodProd = Encomendas.CodProd;
```

```
Select Cliente, Produto, Quant
From Clientes inner join
(Encomendas inner join Produtos
On Produtos.CodProd = Encomendas.CodProd)
On Clientes.CodCli = Encomendas.CodCli;
```

SQL – DML

- **Consultas com três (ou mais) tabelas**

```
Select Cliente, Produto, Quant
From Clientes, Produtos, Encomendas
Where Clientes.CodCli = Encomendas.CodCli
And Produtos.CodProd = Encomendas.CodProd;
```

Resultado – Equi-join com 3 tabelas

Cliente	Produto	Quant
Aníbal	Portátil	10
Belmiro	Portátil	2
Casimiro	Berbequim	5
Aníbal	MS Office	1

CPTG/PSM15 © Porto Editora

SQL – DML

- **Consultas de junção com uso de alias (substituto)**

```
Select Cliente, Produto, Quant
From Clientes, Produtos, Encomendas
Where Clientes.CodCli = Encomendas.CodCli
And Produtos.CodProd = Encomendas.CodProd
And Quant > 5;
```

```
Select Cliente, Produto, Quant
From Clientes C, Produtos P, Encomendas E
Where C.CodCli = E.CodCli
And P.CodProd = E.CodProd
And Quant > 5;
```

SQL – DML

- **Fonte de informação utilizada**
  - **A. A. Azul, Programação e Sistemas de Informação, Porto Editora**